

Fuerza bruta y Backtracking

Autores:

Brayan Santiago González Mateus

José Eduardo Piña Montero

Alejandro Martinez Vivanco

Angel David Quimbayo Carvajal

Profesor:

Carlos Alberto Álvarez Henao

Universidad EAFIT

Estructura de Datos y Algoritmos II

2026

Introducción

Mediante la **Práctica 1** estaremos analizando diferentes tipos de algoritmos que nos ayudarán a resolver múltiples tipos de problemas y a comprender cómo las estrategias utilizadas afectan el costo computacional. En esta práctica utilizaremos dos estrategias de búsqueda y resolución de problemas: **FB (Fuerza Bruta)** y **BT (Backtracking)**.

En la primera parte de esta práctica haremos uso de **FB**, en la cual abordaremos la siguiente pregunta central: **¿cuánto tiempo cuesta encontrar una contraseña probando todo el espacio de búsqueda?** Esto se realizará mediante enumeración sistemática y verificación por hash. Esta estrategia permite encontrar una contraseña a partir de la comparación de los hashes; sin embargo, también permite observar el alto costo computacional que puede generar la aplicación de Fuerza Bruta y el crecimiento exponencial asociado al tamaño del alfabeto y a la longitud de las cadenas.

En la segunda parte de la práctica haremos uso de **BT**, el cual presenta una perspectiva diferente a la Fuerza Bruta. En este caso, se construirán contraseñas de forma incremental de acuerdo con una política de seguridad. Durante la construcción, los estados que no cuenten con una solución factible serán podados, evitando explorar ramas que no pueden conducir a una solución válida. De esta manera, se analizará la complejidad algorítmica y el efecto de la poda al resolver este tipo de problemas mediante Backtracking.

Este trabajo tiene como fin conocer, analizar y comparar cómo las diferentes estrategias de búsqueda influyen en el desempeño de los algoritmos y en el costo computacional necesario para resolver un problema.

El documento estará organizado de la siguiente manera: inicialmente se presenta el contexto del problema y la fundamentación teórica. Luego, se describe el modelamiento de los espacios de búsqueda y las decisiones tomadas para el diseño de los algoritmos, junto con sus respectivos pseudocódigos. Además, se analizarán las decisiones relevantes de implementación, la complejidad temporal y espacial de ambos módulos, los casos de prueba y la experimentación realizada. Finalmente, se presentarán los resultados obtenidos, su análisis y comparación, para concluir con las principales conclusiones del trabajo.

Contexto del problema

Módulo FB – Fuerza Bruta

En este módulo encontraremos una contraseña a partir de un hash conocido. Como los programas normalmente no guardan las contraseñas en forma de texto plano, sino que aplican una función hash para protegerlas, no es posible acceder directamente la contraseña original a partir del hash. así que si limitamos el espacio y lo conocemos podemos calcular el hash y compararlo contra el hash objetivo.

Para esto utilizaremos la estrategia de **FB (Fuerza Bruta)**. Esta estrategia comparará de forma exhaustiva todas las posibles combinaciones dentro de un espacio previamente limitado. A diferencia de otras estrategias, esta no utiliza información adicional para podar, optimizar o descartar candidatos antes de evaluarlos completamente. Sin embargo, garantiza encontrar la solución si esta se encuentra dentro del espacio de búsqueda definido.

Este módulo tiene un límite computacional, y uno de sus principales objetivos es analizar cómo un algoritmo de FB afecta directamente el costo computacional. Además, al aumentar el espacio acotado, como el tamaño del alfabeto o la cantidad de caracteres de la contraseña, la cantidad de posibles combinaciones crece de forma exponencial. La práctica busca precisamente identificar experimentalmente el punto en que este crecimiento deja de ser manejable en un computador personal.

Así que, en conclusión, este módulo nos permite no solo encontrar una contraseña haciendo uso de un algoritmo, sino también analizar cómo aumenta el costo de la búsqueda exhaustiva hasta llegar a un punto en el que el programa puede volverse insostenible para un computador personal. Además, todo el procedimiento se realiza únicamente con contraseñas sintéticas generadas para la práctica, sin aplicarlo a sistemas o credenciales reales.

Contexto del problema

Módulo BT – Backtracking

En este módulo se trabajará con una estrategia totalmente diferente a la anteriormente mencionada, **FB**. Ahora tendremos una política de seguridad para la creación de contraseñas, la cual incluye diferentes restricciones como: cantidad mínima de minúsculas, mayúsculas, dígitos, símbolos, cantidad de caracteres y caracteres consecutivos.

El objetivo es generar contraseñas que cumplan con los requisitos del sistema de creación de contraseñas, evitando crear y analizar todas las posibles combinaciones como normalmente lo haría FB. De esta manera, se busca reducir la cantidad de combinaciones que deben ser exploradas y el trabajo realizado durante la búsqueda.

Durante la creación de las ramas, el sistema deberá ser capaz de analizar, antes de avanzar, si un camino todavía puede llegar a la solución del problema. En caso de que no sea posible, podará esa rama para evitar explorarla por completo y continuará con la próxima.

De esta manera podremos analizar las diferentes características que tiene un algoritmo de **BT** con respecto a un algoritmo de **FB**, aunque esto no necesariamente reduzca por completo la complejidad algorítmica o el tiempo de respuesta, puesto que con este tipo de actividades podemos tener un árbol de búsqueda tan grande que, aun aplicando poda, la complejidad de BT puede seguir siendo muy alta. De hecho, la poda reduce los nodos efectivamente explorados, pero el peor caso del problema puede conservar un comportamiento exponencial.

En conclusión, en este módulo creamos un programa que genera contraseñas con unos límites o parámetros que debe cumplir para poder ser aceptada por el sistema. También podremos analizar cómo la construcción incremental y la acción de **podar** disminuyen la cantidad de nodos explorados, reduciendo así el trabajo innecesario durante la búsqueda.

Fundamentación teórica

Módulo FB– Fuerza Bruta

La fuerza bruta es una estrategia que consiste en encontrar la solución mediante la exploración de todas las posibles combinaciones del problema, sin importar si alguna de esas combinaciones exploradas no es válida para el sistema. En esta práctica, el problema consistía en encontrar una contraseña a partir de un hash SHA-256 objetivo. Para ello, se generaron posibles contraseñas utilizando un alfabeto acotado y un espacio limitado de caracteres. Para resolver el problema, se calculaba el hash de cada una de las posibilidades y se comparaba con el hash objetivo hasta encontrar uno que coincidiera.

Un ejemplo de ello es la instancia *snynkn*, la cual para poderse generar deben pasar por 220236342, un número realmente grande pero que muestra bien la esencia de la fuerza

bruta, que informalmente se podría describir como: *"Genere casos hasta que concuerde con el hash objetivo"*

Fundamentación teórica

Módulo BT – Backtracking

El backtracking es una estrategia para la resolución de problemas en el cual se busca generar todos los casos que vayan cumpliendo con una función o condición que los haga factibles; de lo contrario, se rechazará ese resultado parcial, evitando así que se exploren opciones derivadas de esa misma opción, pudiéndose representar en árboles, teniendo un espacio teórico de tamaño teórico $\sum_{k=0}^n |\Sigma|^k$, ante esto, el peor caso en un algoritmo de backtracking es cuando la política no es muy restrictiva, haciendo que la poda no sea de tanta utilidad, mientras que el mejor caso se da cuando la política es muy restrictiva, haciendo que se puedan descartar ramas desde etapas tempranas que por ende, hará que la poda si sea de mucha utilidad, aunque cabe aclarar que la poda solo es una mejora operativa.

Un ejemplo de esto es la instancia $n=8$ (la cual es mencionada más adelante), la cual es intratable a nivel de computo, ya que tardaría más de 100 años (nuevamente, el tiempo medianamente exacto es mencionado más adelante en el documento), lo cual demuestra que aunque si bien se tenga una poda activa, a la practica es casi igual a fuerza bruta.

Modelamiento

Módulo FB – Fuerza Bruta

Inicialmente, en el programa se tienen las siguientes definiciones: el espacio de búsqueda se encuentra definido por el alfabeto alphabet dentro del programa, como un string compuesto por 36 símbolos, desde la a hasta la z, junto con los dígitos del 0 hasta el 9, como está estipulado en la práctica, Sección 9.1, para el Módulo FB. La longitud del sistema se encuentra entre un mínimo, minLength, con valor de 5, y un máximo, maxLength, con valor de 5. Estas corresponden a las instancias de referencia.

Para las instancias privadas, se ordenaron alfabéticamente los apellidos completos de los integrantes y se realizó la suma de los códigos ASCII de la cadena resultante. El resultado fue 3043; por lo tanto, esta corresponde a nuestra seed. A partir de esta semilla se generaron las siguientes instancias:

Instancia	Candidatos	Contraseña generada	HASH SHA-256	Tiempo de ejecución	Estado
1	52628	czwd	09e296c417f640eb99c6e96af9f24b6f26afcab426a93d0999b2f5b04995d4f5	1845.98 ms	Encontrada
2	33636	az8l	74e463a1755335e25afb25600b1a85b973a500f4d96cddf8870b5e68994502e9	1170.42 ms	Encontrada
3	9311581	ujuns	6816bfc57d2b08814c5cdf160f2142259ba6230357841285eb49efcb44cb5db0	316580 ms	Encontrada
4	16147030	jwdev	0fbb5127afa9afb7730685f079dba760bd821459741a2c5ebaf5a2c23a942e9c	557269 ms	Encontrada
5	220236342	snynkn	c8409f6d0aea4117610bad3fde21df6d0d987f15a43ace3c8b422832cb0857f3	9.40593e+06 ms	Encontrada

Para cada una de las instancias privadas se mantuvo el alfabeto A2, correspondiente a las letras minúsculas de la ‘a’ a la ‘z’ y los números del 0 al 9, teniendo un total de 36 símbolos. La principal variación entre las ejecuciones fue la longitud de las contraseñas, utilizando valores de 4, 5 y 6 caracteres. Para cada instancia, el algoritmo recibe el hash objetivo mediante la variable targetHash y genera todas las posibles combinaciones utilizando el alfabeto definido hasta encontrar una contraseña cuyo hash SHA-256 coincida con el objetivo.

Modelamiento

Módulo BT – Backtracking

El Módulo BT aborda el problema desde el rol de diseñador de una política de contraseñas: dada una política de complejidad (longitud fija, mínimos de tipo de carácter y prohibición de caracteres consecutivos repetidos), el objetivo es generar y contar todas las contraseñas que la cumplen, sin enumerar primero el espacio completo de cadenas y filtrar al final.

Espacio de búsqueda

El alfabeto de la instancia está compuesto por 69 símbolos: 26 minúsculas, 26 mayúsculas, 10 dígitos y 5 símbolos especiales (!, @, #, \$ y %). Nota: se identificó una inconsistencia en el enunciado, donde se indica un total de "69 símbolos", pero la suma real de los cuatro

conjuntos descritos da 67 ($26 + 26 + 10 + 5 = 67$). Esta ambigüedad fue consultada con el docente, y se documenta explícitamente aquí por transparencia (Sección 17 del enunciado).

Estado parcial

Un estado del árbol de búsqueda se representa como el prefijo de la contraseña construido hasta el momento (una cadena de longitud $k \leq n$), acompañado de cuatro contadores que registran cuántos caracteres de cada tipo (minúscula, mayúscula, dígito y símbolo) contiene ese prefijo, y del último carácter agregado (implícito en la propia cadena, usado para verificar la restricción de no repetidos consecutivos).

Estado inicial

La cadena vacía, con los cuatro contadores en cero.

Estados terminales

Cadenas de longitud exacta $n = 8$. Es importante notar que no toda cadena de longitud n es un estado terminal válido: solo aquellas que, además de tener la longitud correcta, satisfacen los cuatro mínimos de la política y no contienen caracteres consecutivos repetidos en ningún punto se consideran soluciones.

Política de la instancia del equipo

La política fue derivada de la semilla 3043, calculada a partir de la concatenación alfabética de los apellidos de los integrantes: $\text{minLower} = 3$, $\text{minUpper} = 2$, $\text{minDigit} = 2$, $\text{minSymbol} = 1$, $n = 8$.

La suma de estos mínimos (8) es exactamente igual a n , lo que implica holgura cero: cualquier posición "desperdiciada" en un tipo de carácter que ya superó su mínimo automáticamente vuelve infactible la rama, sin necesidad de una regla de poda adicional distinta a la fórmula general de factibilidad.

Diseño algorítmico

Módulo FB – Fuerza Bruta

Para el diseño del algoritmo, primordialmente se eligió seguir el pseudocódigo, adaptándolo a la implementación realizada en C++. También hicimos uso de una estructura basada en clases, separando la implementación del algoritmo de fuerza bruta del programa main, donde se encuentran alojadas las diferentes instancias de prueba.

La clase Algorithm, haciendo parte del espacio de nombres BruteForce, contiene la lógica principal del algoritmo y nos permite utilizarlo como un objeto. De esta manera, desde el archivo main.cpp podemos crear un objeto de esta clase y utilizarlo para acceder al método find(), el cual se encarga de ejecutar la búsqueda de la contraseña correspondiente a cada instancia.

Esta separación también permitió mantener organizada la lógica del programa, ya que el main se utiliza principalmente para definir las instancias de prueba, como el targetHash, el alphabet y las longitudes de búsqueda, mientras que la clase Algorithm se encarga de realizar el proceso de Fuerza Bruta. Por esto, para ejecutar una nueva instancia generalmente solo era necesario modificar los valores definidos en el main, sin alterar la lógica principal del algoritmo.

La función find() recibe los parámetros targetHash, alphabet, minLength y maxLength. A partir de estos valores se inicia la búsqueda recorriendo las longitudes definidas y, para cada una de ellas, se llama a la función generateCandidates(). Esta función se encarga de construir las posibles contraseñas utilizando los caracteres del alfabeto y, cuando se completa una contraseña con la longitud indicada, se genera su hash para compararlo con el targetHash.

También utilizamos atributos privados como isFound, candidates y time, los cuales nos permiten guardar información sobre la ejecución del algoritmo. isFound permite saber si se encontró la contraseña, candidates lleva la cuenta de las contraseñas evaluadas y time almacena el tiempo que tomó realizar la búsqueda.

Finalmente, gracias a la separación del código, para realizar las diferentes pruebas únicamente fue necesario cambiar en el main.cpp los valores correspondientes a cada instancia, como el hash objetivo, el alfabeto y la longitud. De esta manera, la lógica del algoritmo se mantuvo igual durante las diferentes ejecuciones y fue más sencillo realizar las pruebas y registrar los resultados.

Diseño algorítmico

Módulo BT– Backtracking

El algoritmo construye cada contraseña de forma incremental, carácter por carácter, evaluando en cada paso si el prefijo parcial todavía puede extenderse hasta una solución válida. Esta evaluación temprana (poda) es lo que distingue a Backtracking de una enumeración exhaustiva pura.

Función de factibilidad

Se diseñaron dos condiciones, evaluadas en cada nodo antes de continuar la exploración:

1. No repetidos consecutivos: si el candidato tiene longitud ≥ 2 y el último carácter agregado coincide con el penúltimo, la rama es infactible.
2. Factibilidad de alcanzar los mínimos: se calcula faltante_total como la suma de lo que aún falta por cumplir en cada uno de los cuatro tipos de carácter (max(0, mínimo – usado) para cada tipo). Si las posiciones libres restantes ($n - \text{longitud_actual}$) son menores que faltante_total, ninguna extensión del prefijo puede satisfacer la política, y la rama se poda.

Se verificó formalmente (y se confirmó empíricamente mediante la implementación) que esta combinación de condiciones garantiza que ninguna cadena de longitud n con algún mínimo incumplido pueda llegar al caso base: en ese punto, las posiciones libres son cero, y cualquier faltante positivo ya habría provocado la poda en un nivel anterior del árbol.

Dos versiones del algoritmo

Se implementaron dos versiones independientes para permitir la comparación exigida en la Sección 8.2:

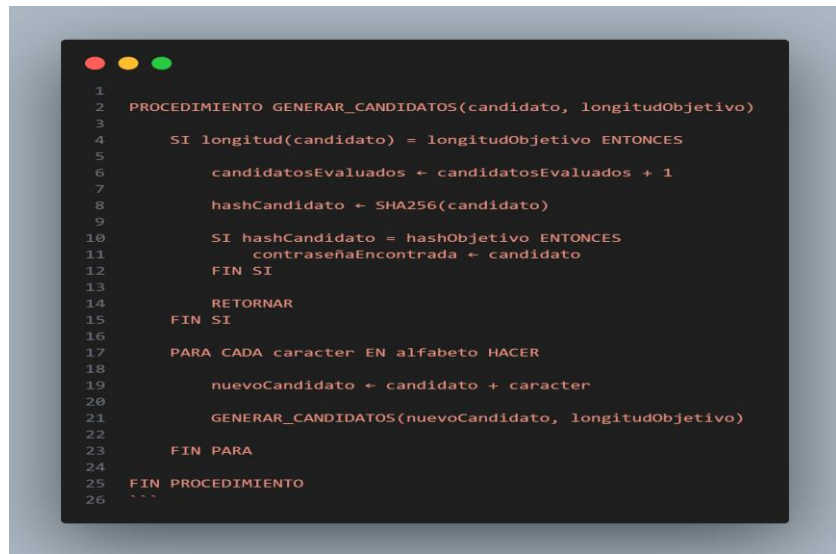
- Con poda: aplica la función de factibilidad en cada nodo del árbol, antes de generar sus descendientes.
- Sin poda: genera el árbol completo hasta las hojas (cadenas de longitud n), sin ningún filtro intermedio, y verifica la política completa (mínimos y no-repetidos-consecutivos) una única vez, al llegar a cada hoja. Esta versión es conceptualmente equivalente a fuerza bruta aplicada sobre el espacio de cadenas de longitud fija, seguida de un filtro final; permite cuantificar cuánto trabajo evita realmente la poda.

Ambas versiones deben producir exactamente el mismo número de soluciones para una misma instancia; esta coincidencia se usa como criterio de verificación de correctitud (ver Sección 10).

Pseudocodigo

Módulo FB– Fuerza Bruta

```
1  ''' text
2  ALGORITMO FuerzaBruta
3
4  ENTRADA:
5      hashObjetivo
6      alfabeto
7      longitudMinima
8      longitudMaxima
9
10 SALIDA:
11     contraseñaEncontrada
12     candidatosEvaluados
13     tiempoEjecucion
14
15 INICIO
16
17     candidatosEvaluados ← 0
18     contraseñaEncontrada ← NO ENCONTRADA
19
20     PARA longitud ← longitudMinima HASTA longitudMaxima HACER
21
22         GENERAR_CANDIDATOS("", longitud)
23
24         SI contraseñaEncontrada ≠ NO ENCONTRADA ENTONCES
25             RETORNAR contraseñaEncontrada
26         FIN SI
27
28     FIN PARA
29
30     RETORNAR contraseñaEncontrada
31
32 FIN
```



Pseudocodigo

Módulo BT– Backtracking

```
ALGORITMO Backtracking

ENTRADA:
    alfabeto
    longitudExacta

SALIDA:
    contraseñasGeneradas
    nodosGenerados
    nodosVisitados
    nodosPodados
    tiempoEjecucion

INICIO

    contraseñasGeneradas ← 0
    nodosGenerados ← 0
    nodosVisitados ← 0
    nodosPodados ← 0
    minLower ← 3
    minUpper ← 2
    minDigit ← 2
    minSimbol ← 1

    usadoLower ← 0
    usadoUpper ← 0
    usadoDigit ← 0
    usadoSimbol ← 0

    inicio ← TIEMPO_ACTUAL()

    BACKTRACKING("", longitud)

    fin ← TIEMPO_ACTUAL()

    tiempoEjecucion ← fin - inicio
```

```

    IMPRIMIR contraseñasGeneradas
    IMPRIMIR nodosGenerados
    IMPRIMIR nodosVisitados
    IMPRIMIR nodosPodados
    IMPRIMIR tiempoEjecucion

FIN

PROCEDIMIENTO BACKTRACKING(candidato, longitudObjetivo)

    nodosGenerados ← nodosGenerados + 1

    SI ES_FACTIBLE(candidato, longitudObjetivo) ES FALSO ENTONCES
        nodosPodados ← nodosPodados + 1
        RETORNAR
    FIN SI

    nodosVisitados ← nodosVisitados + 1

    SI longitud(candidato) = longitudObjetivo Y
        usadoLower >= minLower Y
        usadoUpper >= minUpper Y
        usadoDigit >= minDigit Y
        usadoSimbol >= minSimbol
    ENTONCES

        IMPRIMIR candidato
        contraseñasGeneradas ← contraseñasGeneradas + 1

        RETORNAR

    FIN SI

    PARA CADA caracter EN alfabeto HACER

```

```

    AGREGAR_CONTADORES(caracter)

    BACKTRACKING(candidato, longitudObjetivo)

    candidato.pop_back()

    QUITAR_CONTADORES(caracter)

FIN PARA

FIN PROCEDIMIENTO

PROCEDIMIENTO ES_FACTIBLE(candidato, longitudObjetivo)

    SI tamaño(candidato) >= 2 ENTONCES

        SI candidato[tamaño(candidato) - 2] es igual a candidato[tamaño(candidato) - 1] ENTONCES

            RETORNAR falso

        FIN SI

    FIN SI

    faltante_total ← max(0, minLower - usadoLower)
                    + max(0, minUpper - usadoUpper)
                    + max(0, minDigit - usadoDigit)
                    + max(0, minSimbol - usadoSimbol)

    SI (longitudObjetivo - longitud(candidato)) < faltante_total ENTONCES

        RETORNAR falso

    FIN SI

```

```

    RETORNAR verdadero

FIN PROCEDIMIENTO

PROCEDIMIENTO AGREGAR_CONTADORES(caracter)

    SI caracter es (a,b,c, ...o z) entonces
        usadoLower ← usadoLower + 1
    DE OTRO MODO SI caracter es (A,B,C, ... o Z) entonces
        usadoUpper ← usadoUpper + 1
    DE OTRO MODO SI caracter es (0, 1, 2, ... o 9) entonces
        usadoDigit ← usadoDigit + 1
    DE OTRO MODO SI caracter es (*símbolos*) entonces
        usadoSimbol ← usadoSimbol + 1
    FIN SI

FIN PROCEDIMIENTO

PROCEDIMIENTO QUITAR_CONTADORES(caracter)

    SI caracter es (a,b,c, ...o z) entonces
        usadoLower ← usadoLower - 1
    DE OTRO MODO SI caracter es (A,B,C, ... o Z) entonces
        usadoUpper ← usadoUpper - 1
    DE OTRO MODO SI caracter es (0, 1, 2, ... o 9) entonces
        usadoDigit ← usadoDigit - 1
    DE OTRO MODO SI caracter es (*símbolos*) entonces
        usadoSimbol ← usadoSimbol - 1
    FIN SI

FIN PROCEDIMIENTO

```

***Nota de implementación: * el esqueleto de notación (formato ENTRADA/SALIDA/PROCEDIMIENTO) fue sugerido inicialmente como plantilla de referencia por el integrante del equipo Alejandro Martinez; el contenido de la función de factibilidad, el modelo de estado y las condiciones de poda son de autoría individual de quien desarrolló el Módulo BT.**

Implementación

Módulo FB – Fuerza Bruta

Una de las principales decisiones de implementación fue separar el código en diferentes archivos. Se utilizó `brute_force.hpp` para definir la clase y sus métodos, `brute_force.cpp` para implementar la lógica del algoritmo y `main.cpp` para ejecutar las diferentes instancias. Esto permitió mantener separado el funcionamiento del algoritmo de los datos utilizados en cada prueba.

También se decidió utilizar una clase `Algorithm` para representar el algoritmo de Fuerza Bruta como un objeto. Dentro de esta clase se dejaron como atributos privados `isFound`, `candidates` y `time`, mientras que el método `find()` se dejó como método público para poder ejecutar la búsqueda desde el `main`.

Otra decisión fue dividir el proceso de búsqueda en dos funciones. La función `find()` se encarga de controlar la búsqueda y las longitudes que se deben evaluar, mientras que `generateCandidates()` se encarga de construir las posibles contraseñas de forma recursiva. Esta separación permitió que cada función tuviera una tarea más específica.

Para la comparación de las contraseñas se utilizó la librería `picosha2`, que permite calcular el hash SHA-256 de cada candidato y compararlo con el `targetHash`. Además, se utilizó `chrono` para medir el tiempo que tarda cada ejecución del algoritmo.

```
namespace BruteForce {
    class Algorithm {
    private:
        bool isFound;
        unsigned int candidates;
        double time;
        std::string generateCandidates(
            std::string targetHash,
            std::string alphabet,
            int length,
            std::string currentCandidate
        );
    public:
        std::string find(
```

```

        std::string targetHash,
        std::string alphabet,
        int minLength,
        int maxLength);

    std::string dictionaryAttack(
        std::string targetHash,
        std::string pathDictionary);

    unsigned int getCandidates();
    double getTime();
};
}

```

Nota: lo anterior fue directamente extraído del archivo brute_force.hpp.

También se agregó algo que no estaba en el pseudocódigo, el mini módulo de ataque por diccionario, el cual consiste en traducir cada línea del diccionario a hash y compararla con el hash objetivo.

Implementación

Módulo BT– Backtracking

Clase con variables miembro en vez de struct con parámetros por referencia

Se decidió implementar el estado del algoritmo (contadores de tipo de carácter, parámetros de política y métricas de nodos) como variables miembro de dos clases, BacktrackerConPoda y BacktrackerSinPoda, en vez de un struct pasado por referencia a una función recursiva libre. Esta decisión se tomó por familiaridad previa con el uso de clases en C++ y porque encapsula naturalmente tanto el estado como el comportamiento (los métodos backtrack, esFactible y agregarContadores) en una única unidad.

Parametrización por línea de comandos (argv)

La primera versión de ambas clases definía n y los mínimos de la política como constantes fijas. Esto obligaba a recompilar el programa para cada instancia distinta, lo cual era inviable para la fase de experimentación (Sección 9.2, cinco variantes de dificultad). Se modificó el diseño para recibir estos cinco valores como parámetros del constructor, alimentados desde argv en main.cpp, permitiendo evaluar cualquier instancia sin recompilar: ./ada_p1 .

Separación de la versión con poda y sin poda en clases independientes

En vez de una sola clase con un parámetro booleano que activara o desactivara la poda, se implementaron dos clases separadas, en archivos distintos. Esto mantiene limpia la

comparación de la Sección 8.2 y evita que un error en una condición booleana contamine ambas mediciones, a cambio de cierta duplicación de código (los métodos agregarContadores/quitarContadores son idénticos en ambas clases).

Corrección de desbordamiento de enteros (int a long long)

Durante la fase de experimentación, se observó un valor negativo en el conteo de nodos generados (-882,992,959) para una instancia de política permisiva ($\text{minLower} = \text{minUpper} = \text{minDigit} = \text{minSymbol} = 1$, $n = 6$). La causa fue que las variables de conteo estaban declaradas como int (rango máximo $\approx 2.14 \times 10^9$), y el árbol de búsqueda de esa instancia superó ese límite, produciendo un desbordamiento (overflow) que envolvió el contador a un valor negativo. La corrección consistió en cambiar el tipo de las variables de conteo (nodosGenerados, nodosVisitados, nodosPodados, contraseñasGeneradas) a long long (rango $\approx 9.2 \times 10^{18}$), sin modificar la lógica del algoritmo. Este hallazgo se documenta también como evidencia adicional de la magnitud real del espacio de búsqueda en instancias de baja restricción.

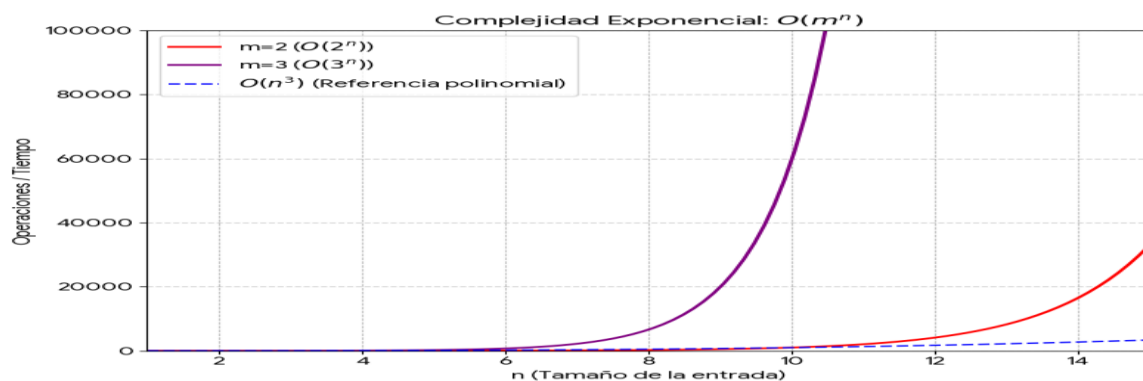
Corrección de un error en el alfabeto

En una versión temprana, el alfabeto contenía una letra k duplicada y omitía la letra q, lo cual duplicaba la exploración de una rama e impedía generar contraseñas con q. Se corrigió a la secuencia completa y sin duplicados de a-z, A-Z, 0-9 y los cinco símbolos definidos.

Análisis de complejidad

Módulo FB – Fuerza Bruta

Para analizar su complejidad, primero revisamos como está estructurado el código, y podemos visualizar que por fuerza bruta es exponencial por estas variable maxLength, ya que esta va a obligar a generar al programa todas las cadenas que estén abajo de ella acotada por minLength, haciendo que con pocos valores vaya a explotar en tiempo, tomando en cuenta también a los alfabetos que son de diferente longitud (26 y 36), este también por lo que sería algo como m^n o en notacion Big O: $O(m^n)$, con grafica:



Mientras que en complejidad espacial sí está bien posicionado con una de $O(1)$, o sea, complejidad lineal, ya que no se usan estructuras de datos para realizar la generación, sino que se genera y de una vez se descarta si no es el buscado.

Análisis de complejidad

Módulo BT– Backtracking

Complejidad temporal — peor caso

El peor caso de Backtracking para este problema conserva la cota exponencial de Fuerza Bruta: cuando la política no impone ninguna restricción efectiva, la poda por mínimos nunca se activa, y el algoritmo recorre un árbol de tamaño comparable al de la enumeración exhaustiva completa, del orden de $|\Sigma| \cdot (|\Sigma| - 1)^{(n-1)}$ nodos. El factor $|\Sigma| - 1$ en los niveles siguientes al primero se debe a la restricción de no repetidos consecutivos, que actúa incluso sin ninguna política de mínimos.

Complejidad temporal mejor caso:

Ocurre cuando la política es tan restrictiva que resulta matemáticamente infactible (la suma de mínimos excede n). La poda detecta la infactibilidad en el nodo raíz mismo, y el algoritmo termina en tiempo $O(1)$ respecto al tamaño del alfabeto y la longitud. Se observó empíricamente en la variante (ii) ($n = 6$, política del equipo, suma de mínimos = 8): 1 nodo generado, 0 ms, frente a 1,634,650,501 nodos y aproximadamente 16.1 minutos de la versión sin poda para la misma instancia.

Caso promedio:

Depende de qué tan pronto, en promedio, un prefijo deja de poder satisfacer la política, es decir, de la holgura entre la suma de mínimos y n . Instancias con holgura cero tienden hacia el peor caso, porque la poda solo se activa cerca del final de cada rama.

Complejidad espacial:

Está dominada por la profundidad del árbol de recursión, $O(n)$. No se almacena el árbol completo en memoria; cada candidato se construye y se deshace incrementalmente.

Evidencia empírica del crecimiento exponencial serie con poda, política fija (minLower = minUpper = minDigit = minSymbol = 1)

n | Nodos generados (con poda) | Tiempo (ms) | Factor de crecimiento respecto a $n-1$

4 | 5,327,841 | 107 |

5 | 634,181,265 | 9,721 | $\times 119.1$ (nodos), $\times 90.8$ (tiempo)

6 | 54,951,581,889 | 951,046 | $\times 86.6$ (nodos), $\times 97.8$ (tiempo)

7 | (no ejecutado) | Estimado >30 horas | Extrapolado desde $n = 6$

Los factores de crecimiento medidos (entre $87\times$ y $119\times$ al aumentar n en una unidad) son consistentes con la cota teórica de $|\Sigma| - 1 = 66$ por nivel adicional del árbol, considerando además el costo extra de imprimir y contar un número creciente de soluciones válidas en cada instancia.

Evidencia experimental adicional: la instancia real del equipo ($n = 8$, holgura cero)

Con la política real del equipo ($\text{minLower} = 3$, $\text{minUpper} = 2$, $\text{minDigit} = 2$, $\text{minSymbol} = 1$), la instancia no pudo ejecutarse completa en un tiempo razonable. Usando el dato real medido en $n = 4$ (20.5 s, 5,327,841 nodos generados con poda, política de holgura cero equivalente en espíritu) y el factor de crecimiento $66^4 \approx 18.97$ millones al pasar de $n = 4$ a $n = 8$, se estima un tiempo superior a 12,000 años incluso con poda activa. Esto constituye evidencia directa de que la poda reduce el trabajo relativo, pero no cambia el orden de complejidad del peor caso, tal como anticipa la Sección 6.2 del enunciado.

Casos de prueba

Módulo FB – Fuerza Bruta

Para la creación y verificación de la semilla y los casos de prueba, se decidió al momento del testing la creación de un script llamado *string_creator.cpp*, el cual nos sirve para poder verificar la semilla y la generación de los casos de prueba para facilitar el trabajo del tester. Siguiendo lo dicho en la sección 9.1 del archivo del enunciado de la práctica, se utilizó el generador congruencial lineal y se utilizó un enfoque recursivo para garantizar la pseudoaleatoriedad del algoritmo.

Casos de prueba

Módulo BT– Backtracking

Cálculo de la semilla

Los apellidos de los integrantes, ordenados alfabéticamente y concatenados en minúsculas, sin espacios ni tildes, son: gonzalezmartinezpinaquimbayo. Se normalizó la letra "ñ" a "n" para el cálculo de la suma de códigos ASCII, dado que el estándar ASCII no incluye caracteres con diacríticos. Esta decisión se documenta explícitamente como resolución de una ambigüedad del enunciado (Sección 17). La suma de códigos ASCII resultante y la semilla obtenida es: 3043.

Parámetros de la política derivados de la semilla (Sección 9.2)

$$\text{minLower} = 2 + (3043 \bmod 3) = 3$$

$$\text{minUpper} = 1 + (3043 \bmod 2) = 2$$

$$\text{minDigit} = 1 + (3043 \bmod 3) = 2$$

$$\text{minSymbol} = 1 \text{ (fijo)}$$

$$n = 8$$

Verificación: $3 + 2 + 2 + 1 = 8 = n$, por lo que se cumple la condición $\leq n$ sin necesidad de realizar ningún ajuste.

Ausencia de la utilidad oficial de verificación

El enunciado (Sección 9) indica que se proveería `resources/verificar_semilla.cpp` como utilidad de referencia para que cada equipo confirmara su semilla. Este archivo no fue publicado por el docente al momento de esta entrega. Se consultó la situación por el canal oficial del curso y se implementó una versión propia (`tests/verificar_semilla.cpp`) que reproduce el mismo procedimiento aritmético descrito en la Sección 9.1/9.2, con el objetivo de permitir la reproducibilidad exigida.

Instancia de referencia común

La instancia de referencia común (Sección 9.2) utiliza los siguientes parámetros: $n = 6$, $\text{minLower} = 2$, $\text{minUpper} = 1$, $\text{minDigit} = 1$, $\text{minSymbol} = 1$, alfabeto completo y sin repetidos consecutivos. Se verificó con la versión con poda un total de 293,922,208 soluciones.

Esta cifra se validó indirectamente mediante un harness de pruebas (`tests/verificar_correctitud.cpp`), que confirmó la coincidencia exacta entre las versiones con y sin poda en cuatro instancias más pequeñas de distinta naturaleza: mínimo en cero, holgura cero, holgura positiva y política vacía. Esto permitió establecer confianza en la correctitud del algoritmo antes de aplicarlo a instancias de mayor tamaño.

Instancia real del equipo

La instancia real del equipo utiliza $n = 8$, $\text{minLower} = 3$, $\text{minUpper} = 2$, $\text{minDigit} = 2$ y $\text{minSymbol} = 1$. Esta instancia fue identificada como computacionalmente intratable debido al costo de exploración del espacio de búsqueda, tal como se analiza en la Sección 9, Análisis de complejidad.

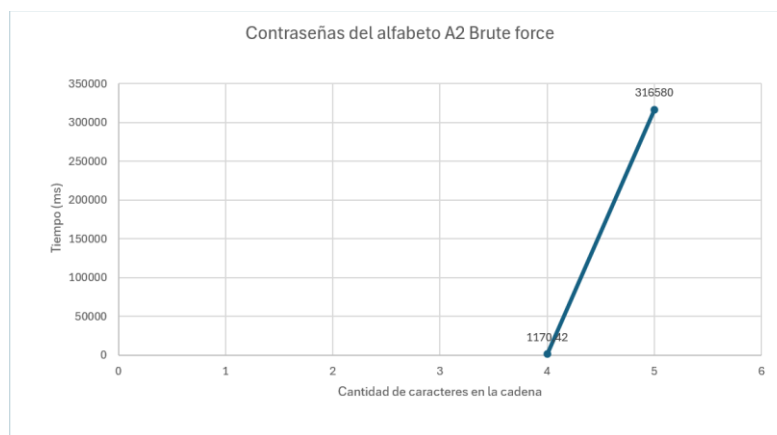
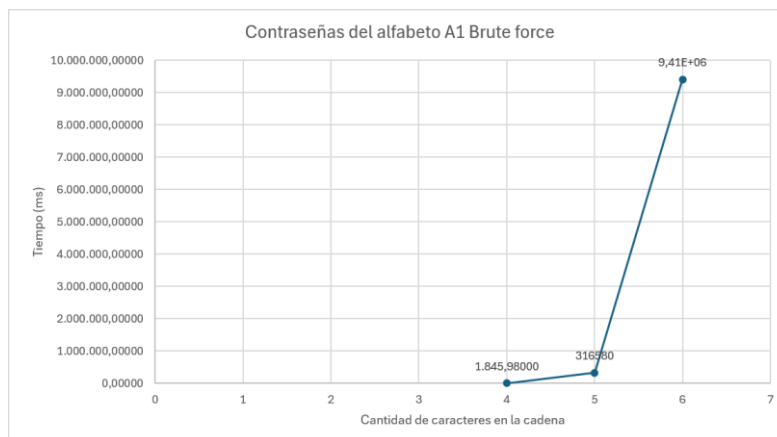
Análisis de resultados

Módulo FB – Fuerza Bruta

En los resultados arrojados por el algoritmo podemos corroborar que en efecto nuestro análisis fue correcto, debido a que aumentando la entrada ligeramente el aumentaba, y en caso de aumentar el exponente ligeramente, el tiempo explotaba, de hecho, como anécdota del trabajo, la primera vez que ejecuté el último caso se demoró 10 horas; después, después de correrlo otras dos veces más el tiempo llegó a converger a 2 horas:

```
sh-5.3$ g++ main.cpp brute_force.cpp -o w
sh-5.3$ ./w
Password found: snykn
Candidates evaluated: 220236342
Execution time: 36425.6 seconds
sh-5.3$
```

Ya siguiendo con las gráficas de los resultados obtenidos:

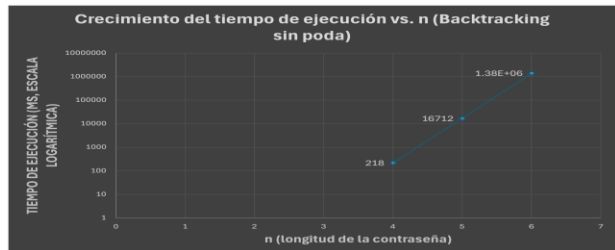
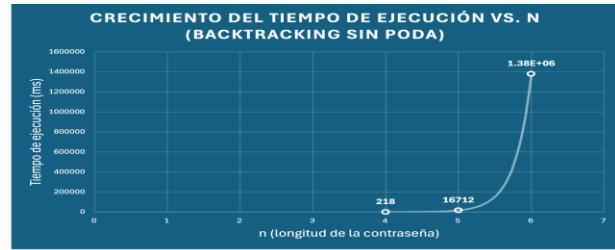
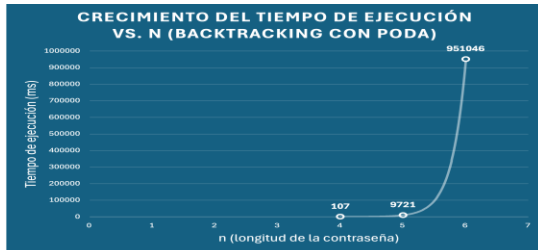


Podemos concluir que nuestro análisis de complejidad temporal fue correcto, ya que se ve que el tiempo explota cuando se cambia ligeramente el exponente, especialmente en la primera gráfica del alfabeto A1.

Análisis de resultados

Módulo BT– Backtracking

Nuevamente, se logró evidenciar que el análisis de su complejidad fue correcto, ya que el algoritmo sin poda explotó, tal como se muestra en la gráfica:



Aunque el algoritmo con poda también explotó, no lo hizo de la forma en la que el algoritmo con poda lo hizo.

Comparación algorítmica

Módulo FB – Fuerza Bruta

La comparación de ataque por diccionario vs fuerza bruta va a ser mil veces mejor ataque por diccionario, ya que como se ven en las gráficas de puntos anteriores, este al tener una complejidad lineal $O(N)$ de tiempo, será muchísimo más rápido que brute force, pero ocurre un pequeño problema, que si en el diccionario no está la contraseña objetivo, este método queda completamente inútil, mientras que brute force sí o sí te va a encontrar la contraseña a costa de que puedas demorar casi medio día.

Comparación algorítmica

Módulo BT– Backtracking

En este módulo, la mejora que tiene el algoritmo con backtracking con poda es muy notoria, ya que aunque sí bien también tiende a explotar, no lo hace de una manera tan agresiva, por así decirlo, como en el algoritmo sin poda. Esto se debe a que se posee una complejidad exponencial, algo parecido a lo ocurrido en fuerza bruta, aunque con una ligera mejora, pero, como se mencionó en clase, es meramente operativa, con un ligero cambio de exponente, esto explota. Sin embargo, lo visto con el algoritmo con poda es algo impresionante, pero aún está lejos de ser algo viable.

Conclusiones

Módulo FB – Fuerza Bruta

Después de realizar el modelamiento, implementación y testeó del código, podemos concluir que los algoritmos tipo Brute force son sumamente poderosos, ya que te encuentran una solución o no (o sea que te garantizan que haya o no una solución para el problema), pero hay dos problemas principales. Uno de esos es el hardware usado para esta prueba, ya que, por ejemplo, si se llega a correr en un procesador Intel Celeron (como lo fue en el caso del tester) el tiempo estimado explota a casi 10 horas, mientras que si se corre en un procesador Ryzen 7 se demora poco menos de 20 minutos; ahora con el segundo problema, es el tiempo, aunque se pueda acelerar con un poco de hardware más potente, siempre va a ser un cuello de botella en esta forma de abordar problemas.

Conclusiones

Módulo BT– Backtracking

En este módulo se pudo ver que este tipo de algoritmos también son muy fuertes, ya que te pueden encontrar una solución sin necesidad de explorar todo el espacio de búsqueda, pero el problema es que no hay mucha diferencia entre la complejidad de fuerza bruta, ya que con un ligero cambio en su exponente su tiempo explota.

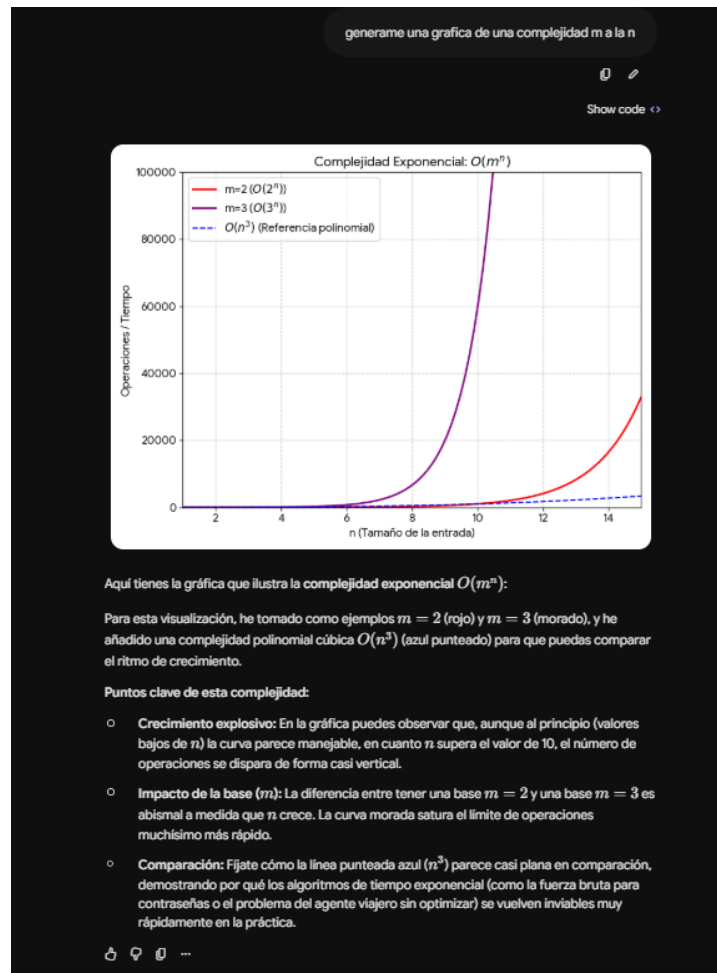
Referencias

- <https://stackoverflow.com/questions/8029121/how-to-hash-stdstring> (aunque no se terminó usando).
- Crédito a los creadores de picosha2.h, que fue parte esencial de este trabajo; es el enlace al repositorio: <https://github.com/okdshin/PicoSHA2>
- <https://www.encryptionconsulting.com/es/education-center/sha-256/>
- Pseudocodigos vistos en el curso

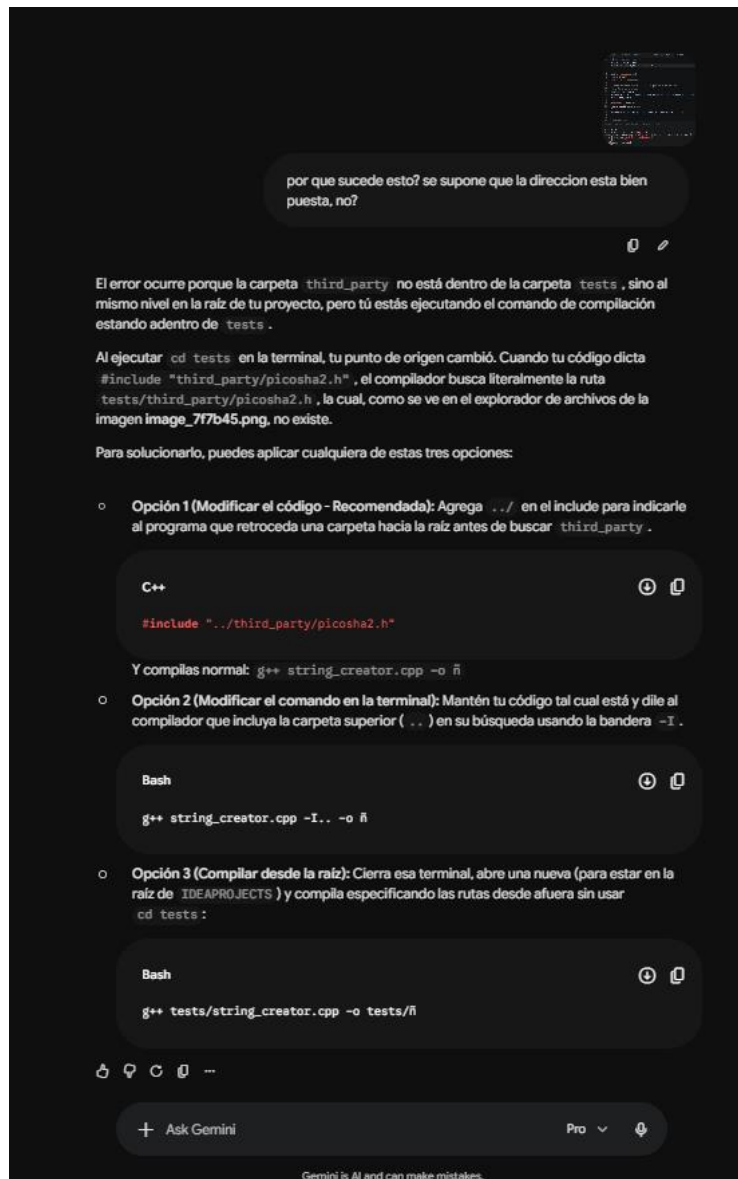
Uso de herramientas de IA:

Gemini:

Sabado 29 de agosto: Se uso para poder generar la gráfica de referencia sobre la complejidad exponencial teórica del algoritmo brute force.



Domingo 30 de agosto: Se usó para pedir una explicación de un error del `string_creator.cpp` ya que al momento en el que se intentó usar la librería para sacar el hash de las pruebas, hubieron errores.



por que sucede esto? se supone que la direccion esta bien puesta, no?

El error ocurre porque la carpeta `third_party` no está dentro de la carpeta `tests`, sino al mismo nivel en la raíz de tu proyecto, pero tú estás ejecutando el comando de compilación estando adentro de `tests`.

Al ejecutar `cd tests` en la terminal, tu punto de origen cambió. Cuando tu código dicta `#include "third_party/picsha2.h"`, el compilador busca literalmente la ruta `tests/third_party/picsha2.h`, la cual, como se ve en el explorador de archivos de la imagen `image_7f7b45.png`, no existe.

Para solucionarlo, puedes aplicar cualquiera de estas tres opciones:

- **Opción 1 (Modificar el código - Recomendada):** Agrega `../` en el include para indicarle al programa que retroceda una carpeta hacia la raíz antes de buscar `third_party`.

```
C++  
#include "../third_party/picsha2.h"
```

Y compilas normal: `g++ string_creator.cpp -o ñ`

- **Opción 2 (Modificar el comando en la terminal):** Mantén tu código tal cual está y dile al compilador que incluya la carpeta superior (`..`) en su búsqueda usando la bandera `-I..`.

```
Bash  
g++ string_creator.cpp -I.. -o ñ
```

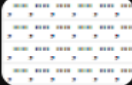
- **Opción 3 (Compilar desde la raíz):** Cierra esa terminal, abre una nueva (para estar en la raíz de `IDEAPROJECTS`) y compila especificando las rutas desde afuera sin usar `cd tests`:

```
Bash  
g++ tests/string_creator.cpp -o tests/ñ
```

+ Ask Gemini Pro

Gemini is AI and can make mistakes.

Lunes 31 de agosto: Se usó para resolver un problema con la tabla de tiempos que estaba en Excel, pero que al pasarlo a CSV ocasionaba problemas de visualización.



por que genera muchas columnas

Esto ocurre porque el programa (Excel o Google Sheets) detectó que hay celdas "usadas" alrededor y debajo de tu tabla. Aunque visualmente parezcan vacías, si alguna celda tiene un espacio en blanco oculto, un cambio de formato (como bordes o tipografía) o fue seleccionada accidentalmente, el exportador asume que forma parte del documento y la rellena con comas para mantener la estructura de la cuadrícula.

Además, la imagen revela dos detalles importantes sobre cómo estaba acomodada tu tabla:

- **La columna A estaba vacía:** Tus filas de datos empiezan con una coma (, "czwdÃ...), lo que indica que dejaste la primera columna en blanco y tu texto realmente empieza en la columna B.
- **Problemas de codificación (Ã):** Ese carácter extraño que aparece después de tus contraseñas es un residuo (espacios de no separación) que se filtra cuando copias texto directamente desde un archivo PDF o una página web.

Para generar un archivo CSV limpio y sin basura, sigue estos pasos:

1. Ve a tu hoja de cálculo original.
2. Selecciona estrictamente el rango de tu tabla, asegurándote de no sombrear filas ni columnas adicionales en blanco.
3. Copia los datos (Ctrl + C).
4. Abre un documento de hoja de cálculo completamente nuevo.
5. Haz clic derecho en la primera celda (A1) y selecciona Pegado especial > Pegar solo valores (o presiona Ctrl + Shift + V). Esto fuerza a que el texto inicie desde el principio y elimina cualquier formato invisible o carácter problemático.
6. Vuelve a exportar o descargar el archivo como .csv .

Cuando abras este nuevo archivo, todas las comillas infinitas y los espacios sobrantes habrán desaparecido.

Contribuciones personales:

Brayan Santiago González Mateus: Fue quien implemento el código de fuerza bruta y redactor del informe en parte de fuerza bruta y backtracking.

José Eduardo Piña Montero: Fue quien implemento el pseudocódigo y código de backtracking, sus respectivos tests, su análisis, realizo la unificación de los módulos y organizo el repositorio.

Alejandro Martínez Vivanco: Fue el creador de los pseudocódigos de fuerza bruta y una parte de backtracking y sus respectivos mds.

Angel David Quimbayo Carvajal: Fue tester del módulo de fuerza bruta, hizo lo respectivo análisis y redactó el informe en la parte de fuerza bruta.